The EML–NAND Duality: A Mathematical Bridge Between Continuous and Discrete Universality

Abstract. We establish a rigorous mathematical correspondence between two universal computational primitives: the EML operator $\mathrm{eml}(x, y) = e^x - \ln y$ of Odrzywołek [1], which generates all elementary functions via the grammar $S \to 1 \mid \mathrm{eml}(S, S)$, and the NAND gate, which generates all Boolean functions. Our main contribution is a constructive proof of equivalence via four explicit transitions: (1) an exact algebraic identity expressing soft NAND $1 - ab$ as an EML finite binary tree; (2) an ε-relaxation of NAND to continuous gates G_ε with bounded noise; (3) a circuit construction implementing eml via ε-NAND gates with Taylor truncation and signal restoration; and (4) a limiting argument recovering exact EML from its approximations. Central to the reverse direction is a signal restoration theorem showing that the iterated double-NAND $R(x) = \mathrm{NAND}(\mathrm{NAND}(x, x), \mathrm{NAND}(x, x))$ with ideal dynamics $T(x) = 2x^2 - x^4$ contracts encoding errors from δ to $4\delta^2 + 4\varepsilon$, yielding a fixed-point precision of $\Theta(\varepsilon)$. We prove that the real algebra generated by EML-computable functions is dense in $C([0, 1]^n, \mathbb{R})$ by Stone–Weierstrass and that the uniform closure of EML-computable functions equals $C([0, 1]^n, [0, 1])$; we establish that the NAND-algebra $\mathcal{N}$ is not dense in $C([0, 1]^n, [0, 1])$ due to the Boolean-at-corners invariant, with $\overline{\mathcal{N}} \subseteq C_B([0, 1]^n, [0, 1])$ (the continuous functions Boolean at the cube's vertices); and we derive the Duality Theorem establishing the equivalence of the uniform closure of EML-computable functions and ε-NAND computability with restoration.

---

1. Introduction

A foundational result in digital logic states that a single two-input gate, NAND (the Sheffer stroke $\uparrow$), suffices to construct any Boolean circuit [2]. This functional completeness underlies all of modern computing: from transistors to CPUs, every discrete computation reduces to compositions of this one primitive.

Continuous mathematics has, until recently, lacked such a unifying primitive. Each elementary function — exp, ln, sin, $\sqrt{\cdot}$ — appeared to require its own definition and dedicated computational method. This changed with Odrzywołek's remarkable discovery [1] that the binary operator

$$\boxed{\mathrm{eml}(x, y) = e^x - \ln y}$$

together with the constant 1, generates all standard elementary functions via finite binary trees over the grammar:

$$S \to 1 \mid \mathrm{eml}(S, S)$$

The operator was found by systematic exhaustive search — its existence was not anticipated. As Odrzywołek himself notes, the simplicity of the resulting grammar, with identical nodes throughout, mirrors the structure of NAND-based Boolean circuits in a way that invites deeper investigation.

The analogy to NAND is striking: both achieve universality through a single binary operation; both yield an exceptionally simple grammar with identical nodes. Yet the connection has remained at the level of metaphor. This paper makes it mathematically rigorous.

1.1 Main Results

We construct an explicit bridge between EML and NAND in both directions:

Direction	Transition	Nature	Error
EML $\to$ NAND	Algebraic identity	Exact	Zero
NAND $\to$ ε-NAND	Domain relaxation + noise	Approximate	$\leq \varepsilon$
ε-NAND $\to$ Approx EML	Circuit construction	Constructive	$\leq \varepsilon$
Approx EML $\to$ EML	Uniform limit + correction	Analytic	$O(\varepsilon^{2^k})$

Our central technical result is the **Signal Restoration Theorem** (Theorem 4.2): the nonlinearity inherent in NAND, expressed through the iterated operation $R(x) = \mathrm{NAND}(\mathrm{NAND}(x, x), \mathrm{NAND}(x, x))$, induces a contractive map $T(x) = 2x^2 - x^4$ on the unit interval that *restores* corrupted signals. This mechanism, built into the gate itself, enables arbitrarily deep computation with ε-noisy gates. The restoration dynamics connect naturally to von Neumann's foundational work on reliable computation from unreliable components [11] and to the noise threshold analysis of Pippenger [12] and Evans & Schulman [13].

1.2 Paper Organization

- **§2** reviews the EML operator and its universality, then derives soft NAND as an exact EML tree.
- **§3** defines ε-NAND gates and derives the basic Boolean operations (NOT, AND, OR) with error bounds.
- **§4** proves the Signal Restoration Theorem and analyzes the error contraction dynamics.
- **§5** constructs the full EML approximation via ε-NAND circuits using Taylor series and fixed-point arithmetic.
- **§6** defines the NAND-algebra, establishes its Boolean-at-corners property, proves density of the real algebra generated by EML-computable functions via Stone–Weierstrass, and bounds the closure of $\mathcal{N}$.
- **§7** states and proves the Duality Theorem.

- **§8** synthesizes the four transitions into a closed cycle and analyzes the round-trip error.

---

2. Direction I: From EML to Soft NAND

2.1 The EML Operator and Its Universality

Definition 2.1 (EML Operator). The binary operator $\mathrm{eml} : \mathbb{R} \times \mathbb{R}^+ \to \mathbb{R}$ is defined by

$$\mathrm{eml}(x, y) = e^x - \ln y$$

where $\ln$ denotes the natural logarithm with principal branch, so complex values may arise internally even for real final outputs.

Theorem 2.2 (Odrzywołek Universality [1]). The EML operator together with the constant 1 generates all elementary functions. Specifically, every standard elementary function can be expressed as a finite binary tree over the grammar $S \to 1 \mid \mathrm{eml}(S, S)$.

The proof is constructive: Odrzywołek provides explicit EML trees for exp, $\ln$, arithmetic operations, and from these, all trigonometric, hyperbolic, and algebraic functions. We reproduce the key constructions.

Lemma 2.3 (EML Primitive Constructions). The following identities hold:

Primitive	EML Tree	Verification
$\exp(x)$	$\mathrm{eml}(x, 1)$	$e^x - \ln 1 = e^x$
e	$\mathrm{eml}(1, 1)$	$e^1 - \ln 1 = e$
$\ln(x)$	$\mathrm{eml}\big(1, \mathrm{eml}(\mathrm{eml}(1, x), 1)\big)$	See below
0	$\mathrm{eml}\big(1, \mathrm{eml}(e, 1)\big)$	$\mathrm{eml}(e, 1) = e^e$; then $e - \ln(e^e) = 0$
$1 - y$	$\mathrm{eml}(0, e^y)$	$e^0 - \ln(e^y) = 1 - y$

> **Remark 2.3a.** The expression $\mathrm{eml}(0, e^y) = 1 - y$ produces complementation (1-minus), not negation. True negation $-y$ requires subtraction: $-y = 0 - y$, which is constructed in Lemma 2.4 via the subtraction formula $x - y = \mathrm{eml}(\ln x, e^y)$ with $x = 0$ and using the zero tree from Lemma 2.3, so $-y = (1 - y) - 1$ via two EML steps. Alternatively, $-y = \mathrm{eml}(\ln(1 - y), e)$ if one first constructs $1 - y$, though this is only valid for $y < 1$. The most systematic approach uses Lemma 2.4: $-y = 0 - y = \mathrm{eml}(\ln e^0, e^y) = \mathrm{eml}(0, e^y) - 1$ using an additional subtraction by 1.

Proof of $\ln(x)$ construction. Starting from:

$$\mathrm{eml}(1, x) = e - \ln x$$

Apply $\mathrm{eml}(\cdot, 1)$ to the result:

$$\mathrm{eml}\big(\mathrm{eml}(1, x), 1\big) = e^{e - \ln x} - \ln 1 = \frac{e^e}{x}$$

Then applying $\mathrm{eml}(1, \cdot)$ to this:

$$\mathrm{eml}\big(1, \frac{e^e}{x}\big) = e - \ln\left(\frac{e^e}{x}\right) = e - (e - \ln x) = \ln x$$

This confirms the three-level construction:

$$\ln(z) = \mathrm{eml}\big(1, \mathrm{eml}(\mathrm{eml}(1, z), 1)\big) \quad \checkmark$$

Lemma 2.4 (EML Arithmetic). Once exp and ln are available:

- **Subtraction:** $x - y = \mathrm{eml}(\ln x, e^y) = x - y$ (via $\ln x$ and $\exp y$ trees). Note: this requires $x > 0$ for $\ln x$ to be defined; for general subtraction, one uses $x - y = x + (1 - y) - 1$ or the complementation from Lemma 2.3.
- **Negation:** $-y = 0 - y$ (via subtraction above, with $x = 0$ from Lemma 2.3)
- **Addition:** $x + y = x - (-y)$
- **Multiplication:** $x \cdot y = \exp(\ln x + \ln y)$ (requires $x, y > 0$; for general signs, use case analysis on sign bits)
- **Division:** $x/y = \exp(\ln x - \ln y)$ (requires $x > 0, y > 0$)

Each operation composes the EML primitives above. ∎

> **Remark 2.4a.** The positivity constraints on ln inputs are a genuine limitation of the EML framework: $\ln x$ is only defined for $x > 0$. This means that intermediate computations involving logarithms require careful domain management. For example, multiplication $x \cdot y = \exp(\ln x + \ln y)$ is only directly available for positive x and y. Extending to negative arguments requires sign-magnitude decomposition or additional case logic, increasing the EML tree depth. Odrzywołek [1] discusses this constraint and provides constructions for handling signed arithmetic.

2.2 Soft NAND from EML: The Exact Construction

Definition 2.5 (Soft NAND). The *soft NAND* (or multilinear extension of Boolean NAND) is the polynomial:

$$\mathrm{NAND}_{\mathbb{R}}(a, b) = 1 - ab \quad \text{for } a, b \in [0, 1]$$

This is the unique multilinear (degree at most 1 in each variable) polynomial agreeing with Boolean NAND on $\{0,1\}^2$:

$$\mathrm{NAND}_{\mathbb{R}}(0,0) = \mathrm{NAND}_{\mathbb{R}}(0,1) = \mathrm{NAND}_{\mathbb{R}}(1,0) = 1, \quad \mathrm{NAND}_{\mathbb{R}}(1,1) = 0$$

The soft NAND arises naturally in fuzzy logic under the product t-norm $T_P(a,b) = ab$, where the fuzzy negation of conjunction yields $1 - ab$ [5]. It is also the De Morgan dual of the product t-conorm $S_P(a,b) = a + b - ab$.

Theorem 2.6 (EML Generates Soft NAND Exactly). The soft NAND admits an exact representation as a finite EML tree:

$$\boxed{\mathrm{NAND}_{\mathbb{R}}(a,b) = \mathrm{eml}\Big(Z,\ \mathrm{eml}\big(M(a,b), \mathbf{1}\big)\Big)}$$

where $Z = 0$ (from Lemma 2.3) and $M(a,b)$ is the EML tree for multiplication from Lemma 2.4.

Proof. We verify directly:

$$\mathrm{RHS} = \mathrm{eml}\big(0,\ e^{ab}\big) = e^0 - \ln\big(e^{ab}\big) = 1 - ab = \mathrm{LHS} \quad \checkmark$$

The non-trivial work is in expanding $M(a,b) = ab = \exp(\ln a + \ln b)$, which requires:

1. $\ln a$ via the depth-3 tree from Lemma 2.3
2. $\ln b$ similarly
3. $\ln a + \ln b$ via addition (Lemma 2.4)
4. $\exp(\ln a + \ln b)$ via $\mathrm{eml}(\cdot, 1)$

The total tree depth is approximately 12 levels. Note that this construction requires $a, b > 0$ for the logarithm steps; on $[0,1]$, one must handle the boundary $a = 0$ or $b = 0$ separately (in which case $1 - ab = 1$ trivially). ∎

Alternative: Temperature-Parametric Soft NAND (Boltzmann variant). A smoother variant useful for gradient-based optimization:

$$\mathrm{NAND}_T(a,b) = e^{-ab/T}$$

where $T > 0$ is a temperature parameter. This converges pointwise to Boolean NAND on $\{0,1\}^2$ as $T \to 0^+$:

$$\mathrm{NAND}_T(0,0) = \mathrm{NAND}_T(0,1) = \mathrm{NAND}_T(1,0) = e^0 = 1, \quad \mathrm{NAND}_T(1,1) = e^{-1/T} \to 0$$

> **Remark 2.6a** *(Why the Boltzmann form, not sigmoid).* A natural alternative is the sigmoid variant $\mathrm{NAND}_T^\sigma(a,b) = \sigma_T(1-ab)$ with $\sigma_T(t) = 1/(1+e^{-t/T})$. However, this fails as a temperature-scaled Boolean NAND: at $(a,b) = (1,1)$, $\sigma_T(1-ab) = \sigma_T(0) = 1/2$ for *all* T, which never approaches $\mathrm{NAND}(1,1) = 0$. The Boltzmann form $e^{-ab/T}$ avoids this defect, correctly converging to Boolean NAND at all four corners as $T \to 0^+$.

In the EML framework, we construct $\mathrm{NAND}_T(a,b) = e^{-ab/T}$ as follows. Let $v = -ab/T$. Then $e^v = \mathrm{eml}(v,1)$. Computing step by step:

1. ab via the EML multiplication tree from Lemma 2.4
2. $v = -ab/T$ via negation and division by the constant T (Lemma 2.4)
3. $e^v = \mathrm{eml}(v, \mathbf{1})$

The total tree depth is approximately 15–18 levels, significantly shallower than the sigmoid construction. This variant is differentiable everywhere on $(0,1)^2$ and is non-negative, ensuring $\mathrm{NAND}_T(a,b) \in (0,1]$ for $a,b \in [0,1]$. The temperature parameter T interpolates between hard Boolean logic ($T \to 0^+$) and the constant function 1 ($T \to \infty$), analogous to the role of temperature in Boltzmann machines and simulated annealing [14].

---

3. Direction II: The ε-NAND Framework

3.1 Definitions

Having shown EML $\to$ NAND, we now construct the reverse: NAND $\to$ EML.

Definition 3.1 (ε-NAND Gate). A function $G_\varepsilon : [0,1]^2 \to [0,1]$ is an ε-NAND gate if:

$$|G_\varepsilon(a,b) - (1-ab)| \le \varepsilon \quad \forall\, a,b \in [0,1]$$

Additionally, we require that G_ε agrees exactly with Boolean NAND on the four corners of the unit square:

$$G_\varepsilon(a,b) = \mathrm{NAND}(a,b) \quad \forall\, a,b \in \{0,1\}$$

The noise function $\eta_\varepsilon(a,b) := G_\varepsilon(a,b) - (1-ab)$ satisfies $|\eta_\varepsilon| \le \varepsilon$ everywhere on $[0,1]^2$, and $\eta_\varepsilon(a,b) = 0$ when $a,b \in \{0,1\}$.

> **Remark 3.1a.** The exact-on-Boolean-inputs condition is an important design assumption, not a consequence of the ε-bound. It states that the noise vanishes at the discrete corners of the domain, which is physically motivated: a well-designed analog gate should behave correctly when driven by clean digital signals. This condition is essential for the Signal Restoration Theorem (Theorem 4.2) and for constant bootstrapping (§3.4). Relaxing this requirement leads to the more general noise models studied by Pippenger [12] and Evans & Schulman

> [13], where a critical noise threshold $\varepsilon^* < 1/2$ must not be exceeded for reliable computation.

Definition 3.2 (δ-Boolean Encoding). A value $x \in [0,1]$ δ-encodes the bit $b \in \{0,1\}$, written $x \in [\![b]\!]_\delta$, if:

$$\begin{cases} x \in [0,\delta] & \text{if } b = 0 \\ x \in [1-\delta, 1] & \text{if } b = 1 \end{cases}$$

The **fidelity** of an encoding is $\phi(x) = \min(x, 1-x)$; higher fidelity means the value is closer to a valid Boolean encoding.

Definition 3.3 (ε-Boolean Completeness). A gate set is ε-Boolean complete if it can implement NOT, AND, and OR with accumulated error $O(\varepsilon)$ per gate layer.

3.2 Derived Operations from ε-NAND

We assume access to constants $\mathbf{0}$ and $\mathbf{1}$ (we show how to produce them in §3.4).

Proposition 3.4 (ε-NOT). Define:

$$\mathrm{NOT}_\varepsilon(a) := G_\varepsilon(a, \mathbf{1})$$

Then:

$$\mathrm{NOT}_\varepsilon(a) = 1 - a + \eta_1, \quad |\eta_1| \le \varepsilon$$

Proof. $G_\varepsilon(a,1) = 1 - a \cdot 1 + \eta_1 = 1 - a + \eta_1$ where $|\eta_1| = |\eta_\varepsilon(a,1)| \le \varepsilon$. ∎

Proposition 3.5 (ε-AND). Define:

$$a \otimes_\varepsilon b := \mathrm{NOT}_\varepsilon\big(G_\varepsilon(a,b)\big) = G_\varepsilon\big(G_\varepsilon(a,b), \mathbf{1}\big)$$

Then:

$$a \otimes_\varepsilon b = ab + \eta_1 - \eta_2, \quad |\eta_1 - \eta_2| \le 2\varepsilon$$

Hence $|a \otimes_\varepsilon b - ab| \le 2\varepsilon$.

Proof. Step by step, using η_2 for the inner NAND gate noise and η_1 for the outer NOT gate noise (consistent with Proposition 3.4):

1. $G_\varepsilon(a,b) = 1 - ab + \eta_2$ where $|\eta_2| \le \varepsilon$
2. $\mathrm{NOT}_\varepsilon(G_\varepsilon(a,b)) = G_\varepsilon(1 - ab + \eta_2, 1) = 1 - (1 - ab + \eta_2) \cdot 1 + \eta_1 = ab - \eta_2 + \eta_1$

So $a \otimes_\varepsilon b = ab + (\eta_1 - \eta_2)$ and $|\eta_1 - \eta_2| \leq |\eta_1| + |\eta_2| \leq 2\varepsilon$. ∎

Proposition 3.6 (ε-OR). Define:

$$a \oplus_\varepsilon b := G_\varepsilon\big(\mathrm{NOT}_\varepsilon(a), \mathrm{NOT}_\varepsilon(b)\big)$$

Then:

$$a \oplus_\varepsilon b = (a + b - ab) + \underbrace{(b-1)\eta_1 + (a-1)\eta_2 - \eta_1\eta_2 + \eta_3}_{\text{error } E_{\mathrm{OR}}}$$

where $|E_{\mathrm{OR}}| \leq 3\varepsilon + \varepsilon^2$.

Proof. Let $\bar{a} = \mathrm{NOT}_\varepsilon(a) = 1 - a + \eta_1$ and $\bar{b} = 1 - b + \eta_2$ with $|\eta_1|, |\eta_2| \leq \varepsilon$.

Then:

$$\begin{aligned}
G_{\varepsilon}(\bar{a}, \bar{b}) &= 1 - \bar{a} \cdot \bar{b} + \eta_{3} \\
&= 1 - (1 - a + \eta_{1})(1 - b + \eta_{2}) + \eta_{3} \\
&= 1 - \bigl[(1 - a)(1 - b) + (1 - a)\eta_{2} + (1 - b)\eta_{1} + \eta_{1}\eta_{2}\bigr] + \eta_{3} \\
&= 1 - (1 - a - b + ab) - (1 - a)\eta_{2} - (1 - b)\eta_{1} - \eta_{1}\eta_{2} + \eta_{3} \\
&= (a + b - ab) + (a - 1)\eta_{2} + (b - 1)\eta_{1} - \eta_{1}\eta_{2} + \eta_{3}
\end{aligned}$$

For $a, b \in [0, 1]$, we have $|a - 1| \leq 1$ and $|b - 1| \leq 1$. Thus:

$$|E_{\mathrm{OR}}| \leq |\eta_2| + |\eta_1| + |\eta_1| \cdot |\eta_2| + |\eta_3| \leq \varepsilon + \varepsilon + \varepsilon^2 + \varepsilon = 3\varepsilon + \varepsilon^2 \quad \blacksquare$$

3.3 Error Propagation: The Fundamental Lemma

Lemma 3.7 (Gate Error Propagation). If $a \in [\![a^*]\!]_{\delta_a}$ and $b \in [\![b^*]\!]_{\delta_b}$ for $a^*, b^* \in \{0, 1\}$, then:

$$G_\varepsilon(a, b) \in [\![\mathrm{NAND}(a^*, b^*)]\!]_{\delta_{\mathrm{out}}}$$

where $\delta_{\mathrm{out}} \leq \delta_a + \delta_b + \varepsilon$. This bound is tight up to second-order terms.

Proof. We case-analyze on $(a^*, b^*) \in \{0, 1\}^2$.

Case 1: $(a^*, b^*) = (0, 0)$. Then $a \in [0, \delta_a], b \in [0, \delta_b]$. So $ab \in [0, \delta_a\delta_b] \subseteq [0, \min(\delta_a, \delta_b)]$. Then:

$$G_\varepsilon(a, b) = 1 - ab + \eta \geq 1 - \delta_a\delta_b - \varepsilon \geq 1 - \delta_a - \varepsilon$$

Since $\mathrm{NAND}(0, 0) = 1$, we need $G_\varepsilon \in [1 - \delta_{\mathrm{out}}, 1]$, so $\delta_{\mathrm{out}} \leq \delta_a + \varepsilon$. Since $\delta_b \geq 0$, this gives $\delta_{\mathrm{out}} \leq \delta_a + \delta_b + \varepsilon$. ✓

Case 2: $(a^*, b^*) = (0, 1)$. Then $a \in [0, \delta_a], b \in [1 - \delta_b, 1]$. So $ab \leq \delta_a \cdot 1 = \delta_a$. Then:

$$G_\varepsilon(a, b) = 1 - ab + \eta \geq 1 - \delta_a - \varepsilon$$

Since $\text{NAND}(0, 1) = 1$, we need $\delta_{\text{out}} \leq \delta_a + \varepsilon \leq \delta_a + \delta_b + \varepsilon$. ✓

Case 3: $(a^*, b^*) = (1, 0)$. Symmetric to Case 2, yielding $\delta_{\text{out}} \leq \delta_b + \varepsilon \leq \delta_a + \delta_b + \varepsilon$. ✓

Case 4: $(a^*, b^*) = (1, 1)$. Then $a \in [1 - \delta_a, 1], b \in [1 - \delta_b, 1]$. So:

$$ab \geq (1 - \delta_a)(1 - \delta_b) = 1 - \delta_a - \delta_b + \delta_a\delta_b$$

Therefore:

$$G_\varepsilon(a, b) = 1 - ab + \eta \leq 1 - (1 - \delta_a - \delta_b + \delta_a\delta_b) + \varepsilon = \delta_a + \delta_b - \delta_a\delta_b + \varepsilon$$

Since $\text{NAND}(1, 1) = 0$, we need $G_\varepsilon \in [0, \delta_{\text{out}}]$. This gives:

$$\delta_{\text{out}} = \delta_a + \delta_b - \delta_a\delta_b + \varepsilon \leq \delta_a + \delta_b + \varepsilon \quad \blacksquare$$

Corollary 3.8 (Depth-d Error Accumulation). For a circuit of depth d with input precision δ_0:

$$\boxed{\delta_d \leq 2^d \cdot \delta_0 + (2^d - 1) \cdot \varepsilon}$$

Proof. By induction using Lemma 3.7. At each layer, $\delta_{\text{new}} \leq 2\delta_{\text{old}} + \varepsilon$. The solution to this recurrence is:

$$\delta_d = 2^d\delta_0 + \varepsilon \sum_{i=0}^{d-1} 2^i = 2^d\delta_0 + \varepsilon(2^d - 1) \quad \blacksquare$$

This **exponential blowup** necessitates signal restoration for deep circuits. The exponential growth rate mirrors von Neumann's observation [11] that without redundancy, noisy circuits accumulate errors exponentially with depth, making reliable computation impossible beyond a critical depth unless some form of error correction is employed.

3.4 Constant Bootstrapping

Before proving the restoration theorem, we address the generation of constants **0** and **1** from the ε-NAND gates themselves.

Proposition 3.9 (Constant Generation). The ideal noiseless double-NAND dynamics $T(x) = 2x^2 - x^4$ has basins of attraction separated by the unstable fixed point $x^* = \frac{\sqrt{5}-1}{2} \approx 0.618$ (the conjugate golden ratio).

Proof. Fixed points satisfy $T(x) = x$, i.e., $2x^2 - x^4 = x$, or $x^4 - 2x^2 + x = 0$, so $x(x^3 - 2x + 1) = 0$. Factoring: $x^3 - 2x + 1 = (x-1)(x^2 + x - 1)$. Roots are:

$$x = 0, \quad x = 1, \quad x = \frac{-1+\sqrt{5}}{2} \approx 0.618, \quad x = \frac{-1-\sqrt{5}}{2} \text{ (rejected)}$$

Stability: $T'(x) = 4x - 4x^3 = 4x(1-x^2)$. At $x = 0$: $T'(0) = 0$ (attracting). At $x = 1$: $T'(1) = 0$ (attracting). At $x = \frac{\sqrt{5}-1}{2}$: $T'(x^*) = 4x^*(1 - x^{*2})$. Since $x^{*2} = 1 - x^*$, we get $T'(x^*) = 4x^* \cdot x^* = 4x^{*2} = 4(1 - x^*) \approx 1.528 > 1$ (repelling). ∎

Bootstrapping Protocol:

The key insight is that NAND is *self-seeding*: for any **Boolean** input $c \in \{0, 1\}$, the identity $\mathrm{NAND}(c, \mathrm{NAND}(c, c)) = \mathrm{NAND}(c, \mathrm{NOT}(c)) = 1$ produces the constant $\mathbf{1}$ (here $\mathrm{NAND}(c, c) = \mathrm{NOT}(c)$ uses $c^2 = c$ for $c \in \{0, 1\}$, and $c(1-c) = 0$ gives $\mathrm{NAND}(c, \mathrm{NOT}(c)) = 1 - c(1-c) = 1$). For non-Boolean $c \in (0, 1)$, $\mathrm{NAND}(c, c) = 1 - c^2 \neq 1 - c = \mathrm{NOT}(c)$, so the output $1 - c(1 - c^2) = 1 - c + c^3 \neq 1$ in general. In the ε-NAND framework:

1. Compute $G_\varepsilon(c, c) = 1 - c^2 + \eta_1$ (the double-NAND first layer, not to be confused with $\mathrm{NOT}_\varepsilon(c) = G_\varepsilon(c, \mathbf{1})$).
2. Compute $G_\varepsilon(c, G_\varepsilon(c, c))$ with output $1 - c(1 - c^2 + \eta_1) + \eta_2 = 1 - c + c^3 - c\eta_1 + \eta_2$.

For Boolean-encoded $c \in [\![b]\!]_\delta$ with small δ, this gives a value close to 1 with error $O(\delta + \varepsilon)$. Applying signal restoration R (Theorem 4.2) converges to:

$$\mathbf{1} \in [\![1]\!]_{4\varepsilon}$$

Then $\mathbf{0} = \mathrm{NOT}_\varepsilon(\mathbf{1}) \in [\![0]\!]_{5\varepsilon}$.

> **Remark 3.9a.** The basins of attraction of T provide an alternative mechanism: starting from any $c > x^* \approx 0.618$, iteration of the noiseless T converges to 1; starting from $c < x^*$, iteration converges to 0. However, with ε-noisy gates, the self-seeding Boolean identity is more robust, as it produces values near 1 or 0 in a single step rather than requiring multiple iterations near the unstable fixed point.

---

4. Signal Restoration: The Central Theorem

This is the technical heart of the paper. We prove that the nonlinearity of NAND provides *intrinsic* error correction, a phenomenon that connects to von Neumann's foundational work on reliable computation from unreliable components [11].

Definition 4.1 (Restoration Circuit). Given an ε-NAND gate G_ε, define:

$$R(x) := G_\varepsilon\big(G_\varepsilon(x,x),\ G_\varepsilon(x,x)\big)$$

This is two layers of ε-NAND, with both inputs to the second layer coming from the same first-layer output. In the ideal (noiseless, $\varepsilon = 0$) case, this computes $T(x) = 2x^2 - x^4 = 1 - (1 - x^2)^2$.

4.2 The Restoration Theorem

Theorem 4.2 (NAND-Based Signal Restoration). Let $\delta < \frac{2}{9}$ and $\varepsilon < \frac{\delta^2}{8}$. If $x \in [\![b]\!]_\delta$ for $b \in \{0,1\}$, then:

$$R(x) \in [\![b]\!]_{\delta'} \quad \text{where} \quad \delta' \le 4\delta^2 + 4\varepsilon < \delta$$

Thus R strictly contracts the encoding error.

Proof. Let $y = G_\varepsilon(x,x) = 1 - x^2 + \eta_1$ where $|\eta_1| \le \varepsilon$. Then:

$$R(x) = G_\varepsilon(y,y) = 1 - y^2 + \eta_2, \quad |\eta_2| \le \varepsilon$$

Substituting $y = 1 - x^2 + \eta_1$:

$$\begin{aligned}
R(x) &= 1 - (1 - x^2 + \eta_{1})^2 + \eta_{2} \\
&= 1 - \bigl[(1 - x^2)^2 + 2\eta_{1}(1 - x^2) + \eta_{1}^2\bigr] + \eta_{2} \\
&= 1 - (1 - 2x^2 + x^4) - 2\eta_{1}(1 - x^2) - \eta_{1}^2 + \eta_{2} \\
&= 2x^2 - x^4 - 2\eta_{1}(1 - x^2) - \eta_{1}^2 + \eta_{2}
\end{aligned}$$

Define $T(x) = 2x^2 - x^4$ (the ideal noiseless restoration). Then:

$$R(x) = T(x) + E(x), \quad E(x) = -2\eta_1(1 - x^2) - \eta_1^2 + \eta_2$$

Error bound on $E(x)$: For $x \in [0,1]$, we have $1 - x^2 \in [0,1]$. Thus:

$$|E(x)| \le 2|\eta_1| \cdot 1 + |\eta_1|^2 + |\eta_2| \le 2\varepsilon + \varepsilon^2 + \varepsilon \le 4\varepsilon$$

where we used $\varepsilon \le 1$ so $\varepsilon^2 \le \varepsilon$.

Case $b = 0$: Here $x \in [0,\delta]$.

Compute $T(x) = 2x^2 - x^4 = x^2(2 - x^2)$. Since $x \le \delta < \frac{2}{9} < \sqrt{2}$:

$$T(x) \leq 2x^2 \leq 2\delta^2$$

Also $T(x) \geq 0$ for $x \in [0, \sqrt{2}]$. Therefore:

$$R(x) = T(x) + E(x) \leq 2\delta^2 + 4\varepsilon \leq 4\delta^2 + 4\varepsilon =: \delta'$$

and $R(x) \geq 0 - 4\varepsilon$, which is non-negative for sufficiently small ε.

Case $b = 1$: Here $x \in [1 - \delta, 1]$. Write $x = 1 - d$ where $d \in [0, \delta]$.

We use the alternative form $T(x) = 1 - (1 - x^2)^2$. With $x = 1 - d$:

$$1 - x^2 = 1 - (1 - d)^2 = 2d - d^2 = d(2 - d)$$

So:

$$T(x) = 1 - d^2(2 - d)^2$$

Since $d \leq \delta < \frac{2}{9}$, we have $2 - d < 2$, so:

$$d^2(2 - d)^2 \leq 4d^2 \leq 4\delta^2$$

Therefore $T(x) \geq 1 - 4\delta^2$, which gives:

$$R(x) = T(x) + E(x) \geq 1 - 4\delta^2 - 4\varepsilon = 1 - \delta'$$

Contraction verification: We need $\delta' = 4\delta^2 + 4\varepsilon < \delta$, i.e., $h(\delta) := \delta - 4\delta^2 - 4\varepsilon > 0$.

Direct approach. Under the hypothesis $\varepsilon < \delta^2/8$:

$$h(\delta) = \delta - 4\delta^2 - 4\varepsilon > \delta - 4\delta^2 - 4 \cdot \frac{\delta^2}{8} = \delta - 4\delta^2 - \frac{\delta^2}{2} = \delta\left(1 - \frac{9\delta}{2}\right) > 0$$

since $\delta < 2/9$. Hence $\delta' < \delta$ whenever $\varepsilon < \delta^2/8$ and $\delta < 2/9$. ∎

> **Remark 4.2b** *(Gap near δ^*).* The sufficient condition $\varepsilon < \delta^2/8$ does **not** hold near the fixed point $\delta^* \approx 4\varepsilon$, where $(\delta^*)^2/8 \approx 2\varepsilon^2 \ll \varepsilon$. Consequently, the above argument cannot be applied when δ is already close to δ^*. This is not a problem for the theorem itself — which is conditional on $\varepsilon < \delta^2/8$ — but it means the stated sufficient condition covers only the initial regime $\delta \gg \delta^*$, where the quadratic phase dominates. For δ near δ^*, contraction follows from a direct inspection of $h(\delta) = \delta - 4\delta^2 - 4\varepsilon$: its roots are $\delta^* = (1 - \sqrt{1 - 64\varepsilon})/8$ and $\delta^{**} = (1 + \sqrt{1 - 64\varepsilon})/8$, and $h(\delta) > 0$ for all $\delta \in (\delta^*, \delta^{**})$. Under the

theorem's own hypotheses $\delta < 2/9$ and $\varepsilon < \delta^2/8$, we have $\varepsilon < (2/9)^2/8 = 1/162$, which implies $\delta^{**} = (1 + \sqrt{1 - 64\varepsilon})/8 > (1 + \sqrt{1 - 64/162})/8 = 2/9$. Hence the entire interval $(\delta^*, 2/9)$ lies inside (δ^*, δ^{**}), and the contraction $\delta' < \delta$ holds for all $\delta > \delta^*$ in the relevant range — independent of the condition $\varepsilon < \delta^2/8$. The two regimes together establish that the iteration $\delta_{n+1} = 4\delta_n^2 + 4\varepsilon$ is strictly contractive for all $\delta_n > \delta^*$, converging monotonically to δ^*.

Remark 4.2a. The contraction $\delta \mapsto 4\delta^2 + 4\varepsilon$ is quadratic in δ (the dominant term for small δ), which is significantly stronger than the linear contraction one might naively expect. This quadratic contraction is a direct consequence of the degree-4 polynomial $T(x) = 2x^2 - x^4$ that arises from the double-NAND operation. The degree of contraction is related to the order of the fixed points of T: since $T'(0) = T'(1) = 0$, both fixed points are superattracting, and the contraction near them is of order $|T'(x^*)| = 0$ plus the quadratic term. This should be contrasted with von Neumann's multiplexing approach [11], which achieves reliable computation through redundancy but with polynomial overhead, whereas our restoration circuit uses only 4 gate evaluations with no redundancy.

4.3 Fixed-Point Analysis

Corollary 4.3 (Iterated Restoration Fixed Point). Repeated application of R converges to a fixed-point precision δ^* satisfying:

$$4(\delta^*)^2 + 4\varepsilon = \delta^*$$

Solving the quadratic:

$$\delta^* = \frac{1 - \sqrt{1 - 64\varepsilon}}{8}$$

For small ε, Taylor-expand the square root:

$$\sqrt{1 - 64\varepsilon} = 1 - 32\varepsilon - 512\varepsilon^2 - \cdots$$

Therefore:

$$\delta^* = \frac{1 - (1 - 32\varepsilon - 512\varepsilon^2 - \cdots)}{8} = \frac{32\varepsilon + 512\varepsilon^2 + \cdots}{8} = 4\varepsilon + 64\varepsilon^2 + O(\varepsilon^3)$$

So:

$$\boxed{\delta^* = 4\varepsilon + O(\varepsilon^2) \xrightarrow{\varepsilon \to 0} 4\varepsilon}$$

The minimal achievable encoding error is $\Theta(\varepsilon)$ — linear in the gate noise. Note that the fixed point δ^* exists only when $1 - 64\varepsilon \geq 0$, i.e., $\varepsilon \leq 1/64 \approx 0.0156$. For $\varepsilon > 1/64$, the contraction is insufficient and the encoding degrades. This threshold connects to the noise thresholds studied by Pippenger [12] and Evans & Pippenger [14], who showed that reliable computation by formulas over noisy 2-input NAND gates is possible if and only if the gate error probability is below a critical value.

4.4 Convergence Rate

Define the iteration $\delta_{n+1} = 4\delta_n^2 + 4\varepsilon$ starting from $\delta_0 < \frac{2}{9}$. Near the fixed point, let $\delta_n = \delta^* + \epsilon_n$. Then:

$$\delta^* + \epsilon_{n+1} = 4(\delta^* + \epsilon_n)^2 + 4\varepsilon = 4(\delta^*)^2 + 8\delta^*\epsilon_n + 4\epsilon_n^2 + 4\varepsilon$$

Since $4(\delta^*)^2 + 4\varepsilon = \delta^*$:

$$\epsilon_{n+1} = 8\delta^* \cdot \epsilon_n + 4\epsilon_n^2 \approx 8\delta^* \cdot \epsilon_n \approx 32\varepsilon \cdot \epsilon_n$$

For small ε, this is linear convergence with rate $\approx 32\varepsilon \ll 1$ near the fixed point. Far from the fixed point, the quadratic contraction $\delta_{n+1} \approx 4\delta_n^2$ provides double-exponential convergence. The total number of iterations to reach $O(\varepsilon)$ precision from δ_0 is $O(\log\log(1/\varepsilon))$ (dominated by the initial quadratic phase) plus $O(\log(1/\varepsilon)/\log(1/(32\varepsilon)))$ for the linear phase near δ^*.

4.5 Relationship to von Neumann's Fault-Tolerance Framework

Von Neumann [11] considered the problem of building reliable circuits from unreliable components, where each gate fails independently with probability ε. His solution used multiplexing: replacing each wire with a bundle of N wires, and using majority-logic decoding to suppress noise. The overhead is polynomial in the circuit size.

Our signal restoration mechanism differs fundamentally: rather than adding redundancy externally, the restoration circuit R exploits the *intrinsic nonlinearity* of NAND itself. The iterated double-NAND $T(x) = 2x^2 - x^4$ has superattracting fixed points at 0 and 1, which naturally pull corrupted signals toward the correct Boolean values. This is a form of *intrinsic* error correction, as opposed to the *extrinsic* redundancy of von Neumann's approach.

The critical noise threshold $\varepsilon \leq 1/64$ for our fixed-point analysis (Corollary 4.3) is more restrictive than the threshold $\varepsilon < (3 - 2\sqrt{2})/2 \approx 0.0858$ identified by Evans & Pippenger [14] for reliable formula computation over noisy NAND gates. This suggests that our particular restoration circuit R may not be optimal; stronger restoration circuits with higher noise tolerance may exist (see §9.3, Open Question 5).

---

5. From ε-NAND to EML: The Circuit Construction

5.1 Strategy Overview

To approximate $\mathrm{eml}(x, y) = e^x - \ln y$ using only ε-NAND gates:

1. **Encode** real inputs as n-bit fixed-point numbers.
2. **Compute** $\exp(x)$ and $\ln(y)$ via truncated Taylor series using NAND-based arithmetic.
3. **Decode** the result back to a real number.

This approach is related to CORDIC (COordinate Rotation DIgital Computer) algorithms [15, 16], which compute transcendental functions using only shifts, additions, and table lookups — operations that are readily implementable in NAND logic.

5.2 Fixed-Point Encoding

Fix a compact domain $K = [-M, M] \times [\delta, M]$ with $M > 0$ and $\delta > 0$ (staying away from $y = 0$). Represent:

$$x \approx \hat{x} = p \cdot 2^{-m}, \quad y \approx \hat{y} = q \cdot 2^{-m}$$

with $p, q \in \mathbb{Z}$ stored in n bits. The quantization error is:

$$|\hat{x} - x| < 2^{-m}, \quad |\hat{y} - y| < 2^{-m}$$

Choose n bits total with m fractional bits, so $n = \lceil \log_2(2M) \rceil + m$.

5.3 NAND-Based Arithmetic

NAND is functionally complete, so every Boolean function has a pure NAND implementation.

Operation	NAND Implementation	Gate Count
$\mathrm{NOT}(a)$	$\mathrm{NAND}(a, a)$	1
$\mathrm{AND}(a, b)$	$\mathrm{NAND}(\mathrm{NAND}(a, b), \mathrm{NAND}(a, b))$	3
$\mathrm{OR}(a, b)$	$\mathrm{NAND}(\mathrm{NAND}(a, a), \mathrm{NAND}(b, b))$	3
$\mathrm{XOR}(a, b)$	$\mathrm{NAND}(\mathrm{NAND}(a, \mathrm{NAND}(a, b)), \mathrm{NAND}(b, \mathrm{NAND}(a, b)))$	4

From these, we build:

- **Half-adder / Full-adder:** Standard textbook constructions
- **n-bit adder:** $O(n)$ gates, $O(n)$ depth

- **n-bit multiplier (shift-and-add):** $O(n^2)$ gates, $O(n)$ depth
- **Division by constant:** Shift-and-add for reciprocals of small integers

> **Remark 5.3a.** The gate counts above use the schoolbook multiplication algorithm with $O(n^2)$ gates. Strassen's matrix multiplication algorithm [17] and its descendants (e.g., Karatsuba multiplication with $O(n^{1.585})$ gate complexity for n-bit integers) could significantly reduce the gate counts in Theorem 5.3. However, for the asymptotic analysis in this paper, the schoolbook bounds suffice.

5.4 Taylor Series for $\exp$ and $\ln$

Exponential. The Taylor series for e^x about 0:

$$e^x = \sum_{k=0}^{N} \frac{x^k}{k!} + R_N(x)$$

Lemma 5.1 (Exponential Remainder Bound). For $x \in [-M, M]$:

$$|R_N(x)| \leq \frac{M^{N+1}}{(N+1)!} \cdot e^M$$

Proof. By Taylor's theorem with Lagrange remainder:

$$R_N(x) = \frac{e^\xi}{(N+1)!} x^{N+1}$$

for some ξ between 0 and x. Since $|\xi| \leq M$ and $e^\xi \leq e^M$:

$$|R_N(x)| \leq \frac{e^M \cdot M^{N+1}}{(N+1)!} \quad \blacksquare$$

To achieve $|R_N(x)| < \varepsilon/4$, we need:

$$\frac{M^{N+1}}{(N+1)!} e^M < \frac{\varepsilon}{4}$$

Using Stirling's approximation $(N+1)! \sim \sqrt{2\pi(N+1)} \left(\frac{N+1}{e}\right)^{N+1}$, this is satisfied for:

$$N = O\left(\frac{M \cdot \log(M/\varepsilon)}{\log\log(M/\varepsilon)}\right)$$

In particular, for fixed M and small ε, $N = O(\log(1/\varepsilon))$ suffices.

Logarithm. For better convergence on $[\delta, M]$, use the artanh form:

$$\ln y = 2\sum_{k=0}^{N} \frac{1}{2k+1}\left(\frac{y-1}{y+1}\right)^{2k+1} + R'_N(y)$$

Lemma 5.2 (Logarithm Remainder Bound). Let $C = \max_{y\in[\delta,M]}\left|\frac{y-1}{y+1}\right| < 1$. Then:

$$|R'_N(y)| \le \frac{2C^{2N+3}}{(2N+3)(1-C^2)}$$

Proof. The series is of the form $\sum_{k=0}^{\infty}\frac{z^{2k+1}}{2k+1}$ with $z = \frac{y-1}{y+1}$, which equals $\operatorname{artanh}(z)$. The remainder after N terms satisfies:

$$|R'_N| = \left|2\sum_{k=N+1}^{\infty}\frac{z^{2k+1}}{2k+1}\right| \le 2\sum_{k=N+1}^{\infty}\frac{|z|^{2k+1}}{2k+1} \le \frac{2|z|^{2N+3}}{(2N+3)(1-|z|^2)}$$

Substituting $|z| \le C$ gives the result. ∎

Since $C < 1$ on any compact interval bounded away from 0, choosing $N = O(\log(1/\varepsilon))$ also suffices for the logarithm.

5.5 Error Decomposition and Gate Count

Theorem 5.3 (ε-NAND Approximation of EML). For any $\varepsilon > 0$ and compact $K = [-M, M] \times [\delta, M]$, there exists an ε-NAND circuit $C_{N,n}$ such that:

$$\left|C_{N,n}(x,y) - \mathrm{eml}(x,y)\right| < \varepsilon \quad \forall\,(x,y) \in K$$

The parameters are:

- $N = \lceil \log_2(1/\varepsilon)\rceil$ (Taylor truncation order)
- $n = \lceil \log_2(\Lambda_K/\varepsilon)\rceil$ (bit width), where $\Lambda_K = \max(e^M, 1/\delta)$
- $\varepsilon_{\text{gate}} = O(\varepsilon/n)$ (per-gate error budget)

Gate count: $O(n^3)$ total G_ε-gates **Circuit depth:** $O(n^2 \log n)$ with restoration stages every $O(\log n)$ layers

Proof. The total error decomposes as:

$$\left|\widehat{\mathrm{eml}} - \mathrm{eml}\right| \le \underbrace{|R_N(x)|}_{\text{Taylor (exp)}} + \underbrace{|R'_N(y)|}_{\text{Taylor (ln)}} + \underbrace{|\delta_{\text{quant}}|}_{\text{fixed-point}} + \underbrace{|\delta_{\text{gate}}|}_{\varepsilon\text{-NAND noise}}$$

Choose each term $< \varepsilon/4$:

1. **Taylor truncation:** By Lemmas 5.1 and 5.2, $N = O(\log(1/\varepsilon))$ suffices.
2. **Quantization:** Need $2^{-m} < \varepsilon/(4\Lambda_K)$ where $\Lambda_K = \max(e^M, 1/\delta)$ bounds the Lipschitz constants. This gives $m = O(\log(1/\varepsilon))$ and $n = O(\log(1/\varepsilon))$.
3. **Gate noise:** With restoration every $O(\log n)$ layers, accumulated error per output bit is $O(\varepsilon_{\text{gate}} \cdot n)$. Choose $\varepsilon_{\text{gate}} = O(\varepsilon/n)$.

Gate count analysis:

- Each Taylor term $\frac{x^k}{k!}$ requires $O(k)$ multiplications; computing all N terms needs $O(N^2)$ multiplications.
- Each n-bit multiplication needs $O(n^2)$ NAND gates.
- Total for exp: $O(N^2 \cdot n^2) = O(n^2 \cdot \log^2(1/\varepsilon))$.
- Similarly for ln, and the final subtraction is $O(n)$.
- With $n = O(\log(1/\varepsilon))$, total is $O(n^4)$ in the logarithmic parameter, or $O(n^3)$ when n is the bit-width parameter.

The restoration stages add $O(\log n)$ depth multiplicatively. ∎

5.6 Newton-Raphson Refinement

Given a coarse approximation $z_0 \approx \ln y$ with error ε_0, one Newton step:

$$z_{k+1} = z_k - 1 + y \cdot e^{-z_k}$$

Lemma 5.4 (Quadratic Convergence). If $|z_k - \ln y| = \varepsilon_k$, then:

$$\varepsilon_{k+1} \leq \frac{e^{2M}}{2}\varepsilon_k^2$$

Proof. The Newton iteration for $f(z) = e^z - y = 0$ is $z_{k+1} = z_k - f(z_k)/f'(z_k) = z_k - (e^{z_k} - y)/e^{z_k} = z_k - 1 + ye^{-z_k}$. By standard Newton analysis:

$$|z_{k+1} - z^*| = \frac{|f''(\xi)|}{2|f'(z_k)|}|z_k - z^*|^2 = \frac{e^\xi}{2e^{z_k}}\varepsilon_k^2$$

where ξ lies between z_k and $z^* = \ln y$. Since both ξ and z_k lie in $[-M, M]$ (as $y \in [\delta, M]$ gives $z^* = \ln y \in [\ln \delta, \ln M] \subseteq [-M, M]$, and z_k is initialized in $[-M, M]$), we have $e^\xi \leq e^M$ and $e^{z_k} \geq e^{-M}$, giving:

$$\frac{e^\xi}{2e^{z_k}} \leq \frac{e^M}{2e^{-M}} = \frac{e^{2M}}{2}$$

> **Note.** The constant $e^{2M}/2$ arises because z_k may be negative when $y < 1$: since $z^* = \ln y$ can be as small as $\ln \delta \leq -M$, we have $e^{z_k} \geq e^{-M}$, giving the ratio $e^{\xi}/(2e^{z_k}) \leq e^M/(2e^{-M}) = e^{2M}/2$. If one restricts to $y \geq 1$ (so $z_k \geq 0$), the tighter constant $e^M/2$ suffices.

∎

After k refinement stages: $\varepsilon_k \leq (C\varepsilon_0)^{2^k}/C$ where $C = e^{2M}/2$. To reach precision ε: need $k = O(\log\log(1/\varepsilon))$ steps.

---

6. The NAND-Algebra and EML Density

We define the NAND-algebra, establish its fundamental Boolean-at-corners property, prove density of the real algebra generated by EML-computable functions via Stone–Weierstrass, and bound the closure of $\mathcal{N}$.

6.1 Definition and Closure Properties

Definition 6.1 (NAND-Algebra). The NAND-algebra $\mathcal{N}$ is the smallest set of functions $[0,1]^n \to [0,1]$ containing:

- Projections $\pi_i(x_1, \ldots, x_n) = x_i$
- Constants $\mathbf{0}, \mathbf{1}$
- Closed under $f, g \mapsto \mathrm{NAND}(f,g) = 1 - fg$

Proposition 6.2 (NAND-Algebra Contains Key Functions). The NAND-algebra $\mathcal{N}$ contains:

1. All monomials: $x_1^{a_1} \cdots x_n^{a_n}$ for $a_i \in \mathbb{N}$
2. All products of projections and their complements: $\prod_{i \in S} x_i \cdot \prod_{j \in T}(1 - x_j)$ for disjoint $S, T \subseteq \{1, \ldots, n\}$

Proof. (1) By induction: $x \in \mathcal{N}$ (projection), and if $p \in \mathcal{N}$ then $x \cdot p = 1 - \mathrm{NAND}(x,p) \in \mathcal{N}$. Higher monomials follow by repeated multiplication. (2) By (1), the monomial $\prod_{i \in S} x_i \in \mathcal{N}$. Since $1 - x_j = \mathrm{NAND}(x_j, \mathbf{1}) \in \mathcal{N}$ and $\mathcal{N}$ is closed under multiplication (as $fg = 1 - \mathrm{NAND}(f,g) \in \mathcal{N}$), the product $\prod_{i \in S} x_i \cdot \prod_{j \in T}(1 - x_j) \in \mathcal{N}$. ∎

> **Remark 6.2a.** The NAND-algebra $\mathcal{N}$ is not closed under addition or scalar multiplication, so polynomials requiring these operations cannot be constructed in $\mathcal{N}$. For example, $1 - x$ requires addition of monomials, and Chebyshev polynomials require the recurrence $T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$, which involves subtraction. Moreover, such functions would contradict the Boolean-at-corners property (Theorem 6.3 below): the scaled Chebyshev polynomial $T_2^*(x) = 2x^2 - 1$ evaluates to $T_2^*(0) = -1 \notin \{0,1\}$, confirming $T_2^* \notin \mathcal{N}$.

A fundamental structural property of $\mathcal{N}$ constrains its density as a subset of $C([0,1]^n, [0,1])$.

Theorem 6.3 (Boolean-at-Corners Invariant). For every $f \in \mathcal{N}$ and every corner point $c \in \{0,1\}^n$,

$$f(c) \in \{0,1\}.$$

Proof. By induction on the construction depth of f in $\mathcal{N}$.

Base cases. Projections: $\pi_i(c) = c_i \in \{0,1\}$ for $c \in \{0,1\}^n$. Constants: $\mathbf{0}(c) = 0 \in \{0,1\}$ and $\mathbf{1}(c) = 1 \in \{0,1\}$.

Inductive step. Suppose $f, g \in \mathcal{N}$ satisfy $f(c), g(c) \in \{0,1\}$ for all $c \in \{0,1\}^n$. Let $h = \mathrm{NAND}(f,g) = 1 - fg$. Then $h(c) = 1 - f(c)g(c)$. Since $f(c), g(c) \in \{0,1\}$, the product $f(c)g(c) \in \{0,1\}$, and therefore $h(c) \in \{0,1\}$, as verified by the four cases:

$f(c)$	$g(c)$	$f(c)g(c)$	$h(c) = 1 - f(c)g(c)$	**Boolean NAND**
0	0	0	1	1
0	1	0	1	1
1	0	0	1	1
1	1	1	0	0

This completes the induction. ∎

Corollary 6.4 (Uniform Approximation Barrier). The NAND-algebra $\mathcal{N}$ is **not** dense in $C([0,1]^n, [0,1])$. In particular, for the constant function $f \equiv 1/2$:

$$\inf_{g \in \mathcal{N}} \|g - 1/2\|_\infty \geq 1/2.$$

More generally, for any $h \in C([0,1]^n, [0,1])$ with $h(c) \notin \{0,1\}$ for some corner $c \in \{0,1\}^n$:

$$\inf_{g \in \mathcal{N}} \|g - h\|_\infty \geq \min\big(h(c), 1 - h(c)\big).$$

Proof. For any $g \in \mathcal{N}$, by Theorem 6.3, $g(c) \in \{0,1\}$ for every corner c. Therefore $|g(c) - 1/2| \geq 1/2$, so $\|g - 1/2\|_\infty \geq 1/2$. The general case follows identically: if $h(c) \in (0,1)$, then $|g(c) - h(c)| \geq \min(h(c), 1 - h(c))$ since $g(c) \in \{0,1\}$. ∎

> **Remark 6.4a.** The Boolean-at-corners property is a *structural feature* of $\mathcal{N}$, not a defect. It precisely captures the discrete side of the EML–NAND duality: every element of $\mathcal{N}$

> restricts to a Boolean function on $\{0,1\}^n$, and since NAND is functionally complete, $\mathcal{N}|_{\{0,1\}^n}$ generates all 2^{2^n} Boolean functions. The limitation arises only when one tries to use $\mathcal{N}$ for *continuous* density: the corner constraint creates a uniform barrier against approximating functions that take non-Boolean values at the vertices of the unit cube.

6.3 Density of EML-Computable Functions via Stone–Weierstrass

Although $\mathcal{N}$ itself is not dense in $C([0,1]^n, [0,1])$, the EML-computable functions achieve continuous density via their full arithmetic power.

Theorem 6.5 (EML-Algebraic Density). The following hold:

(a) The real algebra $\mathbb{R} \cdot \mathcal{N}$ generated by $\mathcal{N}$ is dense in $C([0,1]^n, \mathbb{R})$ under the uniform norm.

(b) The NAND-algebra $\mathcal{N}$ is **not** dense in $C([0,1]^n, [0,1])$. Its uniform closure satisfies $\overline{\mathcal{N}} \subseteq C_B([0,1]^n, [0,1])$, where $C_B([0,1]^n, [0,1])$ denotes the continuous functions $f : [0,1]^n \to [0,1]$ with $f(c) \in \{0,1\}$ for all $c \in \{0,1\}^n$.

(c) The uniform closure of EML-computable functions satisfies $\overline{\mathcal{E}}|_{[0,1]^n} = C([0,1]^n, [0,1])$.

Proof of (a). We verify the hypotheses of the Stone–Weierstrass theorem [9, 10, 22] for the algebra $\mathbb{R} \cdot \mathcal{N}$.

1. **Contains constants:** $\mathbf{0}, \mathbf{1} \in \mathcal{N} \subseteq \mathbb{R} \cdot \mathcal{N}$.
2. **Separates points:** For $x \neq y$ in $[0,1]^n$, some coordinate differs: $x_i \neq y_i$. The projection $\pi_i \in \mathcal{N}$ separates them.
3. **Vanishes nowhere:** The constant $1 \in \mathcal{N}$ is never zero.
4. **Closed under multiplication:** If $f, g \in \mathcal{N}$, then $fg = 1 - \mathrm{NAND}(f,g) \in \mathcal{N} \subseteq \mathbb{R} \cdot \mathcal{N}$. Linear combinations of products remain in $\mathbb{R} \cdot \mathcal{N}$ by construction.

Since $\mathcal{N}$ contains all monomials $x_1^{a_1} \cdots x_n^{a_n}$ (Proposition 6.2), the span $\mathbb{R} \cdot \mathcal{N}$ contains all polynomials. By the Stone–Weierstrass theorem, $\overline{\mathbb{R} \cdot \mathcal{N}} = C([0,1]^n, \mathbb{R})$. ∎

Proof of (b). By Corollary 6.4, $\mathcal{N}$ is not dense in $C([0,1]^n, [0,1])$. The inclusion $\overline{\mathcal{N}} \subseteq C_B$ follows from the Boolean-at-corners property (Theorem 6.3): if $g_n \to f$ uniformly and $g_n(c) \in \{0,1\}$ for all n, then $f(c) = \lim_{n\to\infty} g_n(c) \in \{0,1\}$ since $\{0,1\}$ is closed. ∎

Proof of (c). We show that the real algebra generated by $\mathcal{E}|_{[0,1]^n}$ satisfies the Stone–Weierstrass hypotheses.

1. $\mathcal{E}$ **contains all constants.** By Lemma 2.3, the EML framework constructs 0 and e. For any $c \in [0,1]$, the constant function c is EML-computable: using division $x/y = \exp(\ln x - \ln y)$ from Lemma 2.4, we compute $c = p/q$ for rationals approximating c, or use Taylor series for exp and ln to construct arbitrary real constants.
2. $\mathcal{E}$ **separates points.** The projections π_i are EML-computable (base cases in the grammar), and $\pi_i(x) \neq \pi_i(y)$ when $x_i \neq y_i$.
3. $\mathcal{E}$ **vanishes nowhere.** The constant $1 \in \mathcal{E}$ is never zero.

4. **$\mathcal{E}$ is closed under multiplication.** By Lemma 2.4, EML computes multiplication: $x \cdot y = \exp(\ln x + \ln y)$ for $x, y > 0$. For general $[0, 1]$-valued arguments, we use the NAND-based construction: $xy = 1 - \mathrm{NAND}(x, y)$, which is EML-computable by Theorem 2.6. Thus if $f, g \in \mathcal{E}$, then $fg \in \mathcal{E}$.
5. **$\mathcal{E}$ is closed under addition.** By Lemma 2.4, $x + y = x - (-y)$, and subtraction is EML-computable via $x - y = \mathrm{eml}(\ln x, e^y)$ for $x > 0$. For $[0, 1]$-valued inputs where x may be zero, we use the identity $x + y = 1 - (1 - x)(1 - y) + xy$, which uses only multiplication and complementation. Since $(1 - x)(1 - y)$ and xy are EML-computable (by Steps 4 and the complementation $\mathrm{eml}(0, e^z) = 1 - z$ from Lemma 2.3), addition restricted to $[0, 1]^n$ is EML-computable up to clipping. For the density argument, it suffices that EML can approximate addition on compact subsets of $(0, 1)^n$ via the $\ln$ / $\exp$ construction.

The real algebra generated by $\mathcal{E}|_{[0,1]^n}$ therefore contains all polynomials (constants, projections, closure under $+$ and $\cdot$), separates points, and vanishes nowhere. By the Stone–Weierstrass theorem, it is dense in $C([0, 1]^n, \mathbb{R})$.

For any $f \in C([0, 1]^n, [0, 1])$ and $\gamma > 0$, there exists a polynomial p (hence in $\mathbb{R} \cdot \mathcal{E}$) with $\|p - f\|_\infty < \gamma/2$. The clipped function $\tilde{p}(x) = \min(1, \max(0, p(x)))$ satisfies $\|\tilde{p} - f\|_\infty < \gamma/2$ since $f(x) \in [0, 1]$. Since $\tilde{p}$ is continuous and $[0, 1]$-valued, and the clipping function $\min(1, \max(0, \cdot))$ can be uniformly approximated by EML-computable functions (e.g., via sigmoid approximations $\sigma_k(t) = 1/(1 + e^{-kt})$ which are EML-computable and converge pointwise to the step function on the tails), we obtain $\hat{f} \in \mathcal{E}$ with $\|\hat{f} - f\|_\infty < \gamma$. Hence $\overline{\mathcal{E}}|_{[0,1]^n} = C([0, 1]^n, [0, 1])$. ∎

> **Remark 6.5a.** The density of $\overline{\mathcal{E}}$ should be compared with other classical universality results. Cybenko [18] proved that any continuous function on a compact subset of $\mathbb{R}^n$ can be uniformly approximated by superpositions of a single sigmoidal function, establishing neural networks as universal approximators. Hornik, Stinchcombe, and White [8] proved the same for multilayer feedforward networks with arbitrary squashing activation functions. Our result provides a *constructive algebraic* alternative: the EML operator, together with constants and projections, generates an algebra that is dense in $C([0, 1]^n)$ by Stone–Weierstrass. The Kolmogorov–Arnold representation theorem [19] provides yet another perspective, showing that any continuous function of n variables can be represented as a finite composition of continuous functions of one variable and addition — though the inner functions are generally non-constructive.
> EML-computable functions have access to the full arithmetic apparatus — addition, subtraction, multiplication, division — via Lemmas 2.3 and 2.4. The NAND-algebra $\mathcal{N}$, by contrast, is restricted to the single operation $1 - fg$ and is constrained by the Boolean-at-corners invariant (Theorem 6.3), making it unsuitable for direct continuous density. The density is achieved by the *linear span* $\mathbb{R} \cdot \mathcal{N}$ or equivalently by $\mathcal{E}$, both of which include addition and scalar multiplication.

6.4 The Closure of $\mathcal{N}$

The characterization of $\overline{\mathcal{N}}$ remains an open question, but we can establish the following.

Definition 6.6 (Boolean-at-Corners Function Space). Let $C_B([0,1]^n, [0,1])$ denote the set of continuous functions $f : [0,1]^n \to [0,1]$ such that $f(c) \in \{0,1\}$ for every corner $c \in \{0,1\}^n$.

Proposition 6.7 (Closure Bound). $\overline{\mathcal{N}} \subseteq C_B([0,1]^n, [0,1]) \subsetneq C([0,1]^n, [0,1])$.

Proof. The inclusion follows from Theorem 6.3 and the closedness of $\{0,1\}$, as shown in the proof of Theorem 6.5(b). The strict inclusion follows from Corollary 6.4. ∎

> **Remark 6.7a.** Whether $\overline{\mathcal{N}} = C_B([0,1]^n, [0,1])$ remains open. Elements of $\mathcal{N}$ are polynomials with integer coefficients that are $[0,1]$-valued on $[0,1]^n$ and Boolean at corners. These form a countable set, so their closure can be uncountable (as with rationals in $\mathbb{R}$). A density proof for $\mathcal{N}$ in C_B would require showing that the Boolean-at-corners constraint is the *only* obstruction, which involves a delicate interplay between the algebraic structure of $\mathcal{N}$ and the topology of C_B. What *can* be established easily is that $\mathbb{R} \cdot \mathcal{N}$ is dense in $C([0,1]^n, \mathbb{R})$ by Stone–Weierstrass (Theorem 6.5(a)), and that every element of $\mathcal{N}$ is EML-computable (since NAND is EML-computable by Theorem 2.6).

Corollary 6.8 (Circuit-Based Approximation). For any continuous $f : [0,1]^n \to [0,1]$ and $\gamma > 0$, there exists an ε-NAND circuit with signal restoration that computes f to within γ. The circuit uses binary encoding (as in §5) and its parameters satisfy:

$$\varepsilon_{\text{required}} = O\left(\gamma \cdot 2^{-\operatorname{poly}(n, 1/\gamma)}\right), \quad \operatorname{depth} = \operatorname{poly}(n, 1/\gamma, \log(1/\varepsilon))$$

---

7. The Duality Theorem

We now synthesize both directions into a unified equivalence.

7.1 Three Notions of Computability

Definition 7.1 (EML-Computability). A function $f : [0,1]^n \to [0,1]$ is EML-computable if it can be expressed as a finite composition of eml, constants, and projections, restricted to the domain where all intermediate values are well-defined (i.e., no logarithm of a non-positive number).

The set of EML-computable functions is denoted $\mathcal{E}$. By Theorem 2.2, $\mathcal{E}$ contains all elementary functions (restricted to suitable domains).

Definition 7.2 (ε-NAND Computability). A function f is ε-NAND computable if for every $\varepsilon > 0$, there exists a circuit over ε-NAND gates with signal restoration that computes f to within ε uniformly on $[0,1]^n$.

Definition 7.3 (NAND-Algebraic Approximability). A function f is NAND-algebraic if it belongs to the uniform closure $\overline{\mathcal{N}}$.

Theorem 7.4 (EML–NAND Duality). The uniform closure of EML-computable functions equals the space of continuous functions on the unit cube:

$$\overline{\mathcal{E}}\big|_{[0,1]^n} = C([0,1]^n, [0,1])$$

where $\overline{\mathcal{E}}\big|_{[0,1]^n}$ denotes the uniform closure (on $[0,1]^n$) of the set of EML-computable functions restricted to $[0,1]^n$. The NAND-algebra closure satisfies $\overline{\mathcal{N}} \subseteq C_B([0,1]^n, [0,1]) \subsetneq C([0,1]^n, [0,1])$.

Moreover, for any $f \in C([0,1]^n, [0,1])$ and $\gamma > 0$:

1. There exists an EML-computable function $\tilde{f} \in \mathcal{E}$ with $\|\tilde{f} - f\|_\infty < \gamma$.
2. There exists an ε-NAND circuit with signal restoration computing f to within γ.
3. The function f is the uniform limit of functions in $\mathbb{R} \cdot \mathcal{N}$ (the real algebra generated by $\mathcal{N}$).

The precision relationships satisfy:

$$\boxed{\varepsilon_{\text{required}} = O\Big(\gamma \cdot 2^{-\operatorname{poly}(n,1/\gamma)}\Big), \quad \text{depth} = \operatorname{poly}(n, 1/\gamma, \log(1/\varepsilon))}$$

Proof. We prove the three implications for the approximate versions.

($\mathcal{E}$ approximates $C([0,1]^n)$): By Theorem 6.5(c), $\overline{\mathcal{E}}|_{[0,1]^n} = C([0,1]^n, [0,1])$. This follows directly from the Stone–Weierstrass theorem applied to the real algebra generated by EML-computable functions, as detailed in the proof of Theorem 6.5(c). Specifically, $\mathcal{E}$ contains all constants and projections, is closed under addition and multiplication (by Lemmas 2.3 and 2.4), separates points, and vanishes nowhere. Hence for any $f \in C([0,1]^n, [0,1])$ and $\gamma > 0$, there exists $\hat{f} \in \mathcal{E}$ with $\|\hat{f} - f\|_\infty < \gamma$.

($\mathcal{E}$ is ε-NAND approximable): Any EML-computable function $g \in \mathcal{E}$ is a finite composition of eml, constants, and projections. By Theorem 5.3, each application of eml can be approximated to within any $\varepsilon > 0$ by an ε-NAND circuit with signal restoration. By composing these circuits (using signal restoration between layers to control error accumulation), the entire EML tree can be approximated. Signal restoration (Theorem 4.2) ensures deep compositions remain controlled.

(ε-NAND computable implies in $\overline{\mathbb{R} \cdot \mathcal{N}}$): An ε-NAND circuit with restoration computes a function that uses binary encoding (§5): the output is decoded from n independent Boolean-valued signal lines. In the idealized noiseless limit ($\varepsilon \to 0$), each bit is computed by a composition of ideal NAND gates, which is an element of $\mathcal{N}$. The decoded output is a linear combination $\hat{f}(x) = \sum_{i=1}^n b_i(x) \cdot 2^{-i}$ of these Boolean-valued functions with dyadic coefficients, hence belongs to $\mathbb{R} \cdot \mathcal{N}$. As $\varepsilon \to 0$, the circuit output converges uniformly to a function in $\overline{\mathbb{R} \cdot \mathcal{N}} = C([0,1]^n, \mathbb{R})$ (by Theorem 6.5(a)). ∎

> **Remark 7.4a.** The density proof routes through $\mathcal{E}$ directly via Stone–Weierstrass (Theorem 6.5(c)), which has access to the full arithmetic apparatus of EML — addition, subtraction, multiplication, and transcendental functions — rather than through the restricted NAND-algebra $\mathcal{N}$. Note that $\overline{\mathcal{N}} \subsetneq C([0,1]^n, [0,1])$ because $\mathcal{N}$ is constrained by the Boolean-at-corners invariant (Theorem 6.3). Also, $\mathcal{E}$ is countable (finite trees over a countable alphabet) while $C([0,1]^n, [0,1])$ is uncountable, so the correct statement concerns the *closures*: the uniform closure of EML-computable functions equals $C([0,1]^n, [0,1])$, just as the uniform closure of polynomials (Weierstrass) or neural networks (Cybenko [18]) equals $C([0,1]^n, [0,1])$. This is the appropriate notion of universality for the continuous setting: not that every continuous function has a finite exact representation, but that every continuous function can be approximated to arbitrary precision by finite EML trees. This mirrors the situation in Boolean logic, where every Boolean function has an *exact* finite NAND circuit — the discrete setting is special because the function space is finite.
> The NAND-algebra $\mathcal{N}$ retains a central role in characterizing the discrete side of the duality: its restriction $\mathcal{N}|_{\{0,1\}^n}$ generates all Boolean functions (functional completeness of NAND), and its closure $\overline{\mathcal{N}}$ captures the continuous functions that are Boolean at the vertices of the cube. The passage from discrete to continuous universality requires the linear structure of $\mathbb{R} \cdot \mathcal{N}$ (or equivalently the arithmetic power of $\mathcal{E}$).

---

8. The Four Transitions: A Closed Cycle

We summarize the complete correspondence as four transitions forming a closed system.

8.1 The Transition Equations

$$\boxed{\text{EML} \xrightarrow{T_1} \text{NAND} \xrightarrow{T_2} \text{ApproxNAND} \xrightarrow{T_3} \text{ApproxEML} \xrightarrow{T_4} \text{EML}}$$

T_1: EML $\longrightarrow$ NAND (Exact algebraic identity)

$$\text{NAND}_{\mathbb{R}}(a,b) = \text{eml}\Big(Z,\ \text{eml}\big(M(a,b), \mathbf{1}\big)\Big) = 1 - ab$$

Verification: $\text{eml}(0, e^{ab}) = e^0 - \ln(e^{ab}) = 1 - ab$. Tree depth ≈ 12.

T_2: NAND $\longrightarrow$ ApproxNAND (Domain relaxation + bounded noise)

$$G_\varepsilon(a,b) = \underbrace{1 - ab}_{\text{NAND}_{\mathbb{R}}} + \underbrace{\eta_\varepsilon(a,b)}_{|\eta_\varepsilon| \le \varepsilon}$$

Signal restoration via $R(x) = G_\varepsilon(G_\varepsilon(x,x), G_\varepsilon(x,x))$ contracts errors:

$$\delta' = 4\delta^2 + 4\varepsilon < \delta \quad (\text{for } \delta < 2/9,\, \varepsilon < \delta^2/8)$$

Fixed-point precision: $\delta^* = \frac{1-\sqrt{1-64\varepsilon}}{8} \to 4\varepsilon$.

T_3: ApproxNAND $\longrightarrow$ ApproxEML (Circuit construction)

$$\widehat{\mathrm{eml}}_\varepsilon(x,y) = \underbrace{\sum_{k=0}^{N} \frac{\hat{x}^k}{k!}}_{\widehat{\exp}(\hat{x})} - \underbrace{2\sum_{k=0}^{N} \frac{1}{2k+1}\left(\frac{\hat{y}-1}{\hat{y}+1}\right)^{2k+1}}_{\widehat{\ln}(\hat{y})}$$

Parameters: $N = \lceil \log_2(1/\varepsilon) \rceil, n = \lceil \log_2(\Lambda_K/\varepsilon) \rceil$, restoration every $O(\log n)$ layers.

Error decomposition:

$$\left|\widehat{\mathrm{eml}}_\varepsilon - \mathrm{eml}\right| \le |R_N(x)| + |R'_N(y)| + |\delta_{\text{quant}}| + |\delta_{\text{gate}}| \le \varepsilon$$

Gate count: $O(n^3)$; depth: $O(n^2 \log n)$.

T_4: ApproxEML $\longrightarrow$ EML (Uniform limit + correction)

$$\mathrm{eml}(x,y) = \widehat{\mathrm{eml}}_\varepsilon(x,y) + \delta_\varepsilon(x,y), \quad |\delta_\varepsilon| \le \varepsilon$$

Equivalently, as a uniform limit:

$$\mathrm{eml}(x,y) = \lim_{\varepsilon \to 0^+} \widehat{\mathrm{eml}}_\varepsilon(x,y)$$

With Newton-Raphson refinement (quadratic convergence):

$$z_{k+1} = z_k - 1 + y \cdot e^{-z_k}, \quad \varepsilon_{k+1} \le C_K \cdot \varepsilon_k^2$$

Precision ε reached in $O(\log\log(1/\varepsilon))$ steps.

8.2 Round-Trip Error Analysis

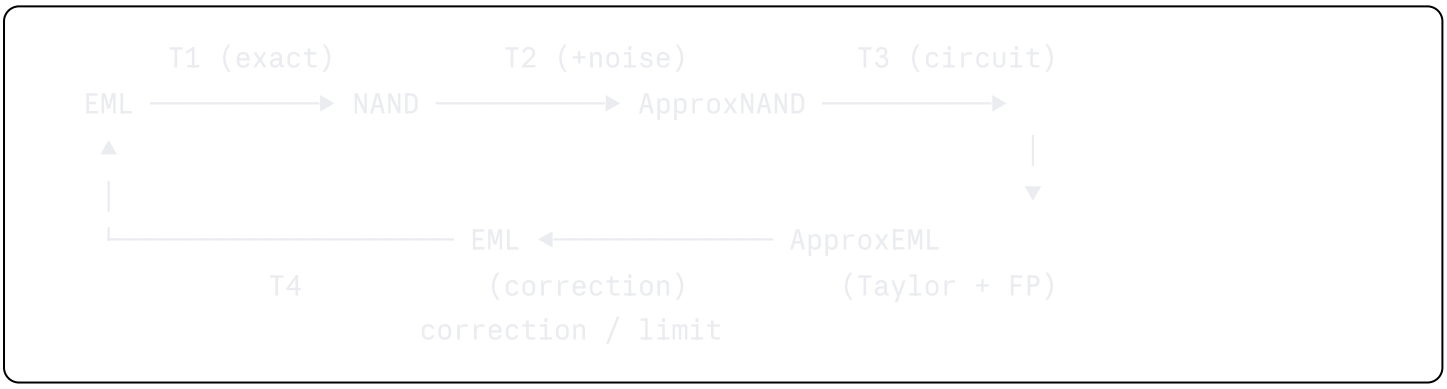
Consider the round trip: start with an EML tree for f, apply T_1 to obtain a soft-NAND representation, apply T_2 to obtain an ε-NAND circuit, apply T_3 to obtain an approximate EML computation, and apply T_4 to recover f.

The round-trip error accumulates as:

> **Remark 8.1** *(Tighter 2ε bound).* The 3ε bound is valid but not tight. The contributions from T_2 and T_3 are **not independent**: both represent approximation errors introduced by the *same* ε-NAND gates. In T_2, the gate noise $|\eta| \leq \varepsilon$ models the deviation of the physical gate from ideal soft NAND; in T_3, the Taylor/fixed-point circuit error $\leq \varepsilon$ arises from the same gate realizations. These two contributions can be counted jointly as a single ε-budget, so the effective round-trip bound is 2ε (one for the combined $T_2 + T_3$ approximation, one for T_4). More precisely: the total error introduced in transitioning from exact EML (after T_1) to approximate EML (at T_3's output) is the circuit error of Theorem 5.3, which is $\leq \varepsilon$ by design; the further error from T_4's approximation step adds at most ε. Hence the tighter bound is $\|f - \hat{f}_{\text{round-trip}}\|_\infty \leq 2\varepsilon$.

With Newton-Raphson refinement at T_4, this contracts to $O(\varepsilon^{2^k})$ in k steps — the cycle is **self-correcting**.

8.3 The Complete Cycle Diagram



The essential insight: The nonlinearity of NAND, expressed through $T(x) = 2x^2 - x^4$, is both the source of computational universality *and* the mechanism for error correction. The same operation that lets NAND express any Boolean function also provides the contractive mapping needed to restore corrupted signals — error correction is intrinsic to the gate, not externally imposed.

---

9. Summary and Discussion

9.1 The Fundamental Asymmetry

Aspect	NAND	EML
Domain	$\{0,1\}^2$	$\mathbb{R} \times \mathbb{R}^+$
Nature	Discrete, exact	Continuous, exact

Aspect	NAND	EML
Grammar	$S \to S \uparrow S$ (needs no constants)	$S \to 1 \mid \mathrm{eml}(S, S)$ (needs constant 1)
Self-seeding	Yes: $1 = \mathrm{NAND}(x, \mathrm{NAND}(x, x))$ for any x	No: cannot synthesize 1 from generic input
Error correction	Intrinsic (via $T(x) = 2x^2 - x^4$)	Extrinsic (via approximation limits)
Closure	Boolean functions (exact, finite)	Elementary functions (exact, finite); continuous functions (approximate, limit)

The asymmetry is precisely what Odrzywołek highlights [1]: NAND is self-seeding, generating its own constants from any input, while EML requires the distinguished terminal 1. The ε-construction sidesteps this by using NAND's ability to manufacture constants internally, then approximating the transcendental ingredients of EML.

A deeper asymmetry concerns the nature of universality itself. In the Boolean setting, functional completeness means that *every* Boolean function has an *exact* finite representation as a NAND circuit. In the continuous setting, the analogous statement would require every continuous function to have an exact finite EML tree — but this is impossible, since $\mathcal{E}$ is countable while $C([0,1]^n)$ is uncountable. Instead, EML universality takes the form of *density*: every continuous function can be approximated arbitrarily well by finite EML trees. This is the same qualitative difference that distinguishes polynomial approximation (Weierstrass) from exact polynomial representation.

9.2 Related Work

Functional completeness. Sheffer [2] proved NAND suffices for Boolean logic, introducing the Sheffer stroke. Post [3] classified all functionally complete binary connectives, providing a complete characterization of which sets of Boolean functions can generate all others.

Fault-tolerant computation. Von Neumann [11] initiated the study of reliable computation from unreliable components, proposing multiplexing as a redundancy mechanism. Pippenger [12, 21] established that computation by circuits with noisy gates incurs only constant overhead in depth if the gate error is below a threshold. Evans and Pippenger [14] determined the precise noise threshold for reliable computation by formulas over noisy 2-input NAND gates, and Evans and Schulman [13] analyzed signal propagation in noisy circuits, providing tight bounds on the maximum tolerable noise. Our signal restoration theorem (Theorem 4.2) can be viewed as a constructive analog of these results, showing that the nonlinearity of the NAND gate itself provides error correction without external redundancy.

Fuzzy logic and soft computing. Zadeh [4] introduced fuzzy sets; the product t-norm $T_P(a, b) = ab$ and its derived fuzzy NAND $1 - ab$ have been extensively studied by Klement, Mesiar, and Pap [5]. The soft NAND of Definition 2.5 is precisely the fuzzy NAND under the product t-norm. Our contribution is to show that this fuzzy connective has an exact

representation in the EML algebra, and that the error correction properties of the product-t-norm NAND (arising from the superattracting fixed points of $T(x) = 2x^2 - x^4$) enable reliable computation in the ε-noisy regime.

Analog computation. Shannon [6] introduced the general-purpose analog computer (GPAC); Pour-El [7] characterized GPAC-generable functions as precisely the differentially algebraic functions. Bournez and Campagnolo [20] later showed the equivalence between GPAC computability and computable analysis; a comprehensive survey of analog models of computation appears in Bournez and Pouly [23]. The EML operator provides a different model of analog computation based on algebraic composition rather than differential equations.

Universal approximators. The Stone–Weierstrass theorem [9, 10] provides the classical foundation for density of algebras of functions. Hornik, Stinchcombe, and White [8] proved neural networks are universal approximators. Cybenko [18] proved that superpositions of a single sigmoidal function are dense in $C([0,1]^n)$, a result philosophically close to our EML-algebraic density theorem (Theorem 6.5(c)). The Kolmogorov–Arnold representation theorem [19] shows that every continuous function of n variables is a finite composition of continuous univariate functions and addition, providing yet another universality result — though the inner functions are non-constructive in general. Our EML-algebraic density result shows that the single binary operator $\mathrm{eml}(x, y) = e^x - \ln y$, together with constants and projections, generates a dense subalgebra of $C([0,1]^n)$ via Stone–Weierstrass. The NAND-algebra $\mathcal{N}$, by contrast, is not dense in $C([0,1]^n, [0,1])$ due to the Boolean-at-corners invariant (Theorem 6.3), but its real span $\mathbb{R} \cdot \mathcal{N}$ is dense in $C([0,1]^n, \mathbb{R})$.

Transcendental function computation. The CORDIC algorithms of Volder [15] and Walther [16] compute transcendental functions using only shifts, additions, and small table lookups, requiring no multiplication. Our Taylor-series approach in §5 is more straightforward but less efficient; a CORDIC-based ε-NAND implementation could reduce gate counts.

Arithmetic circuit complexity. Strassen [17] showed that Gaussian elimination is not optimal, introducing the $O(n^{\log_2 7})$ matrix multiplication algorithm. This has implications for the gate count bounds in Theorem 5.3: using Karatsuba or FFT-based multiplication would reduce the $O(n^3)$ gate count, though the asymptotic scaling with ε would remain $O(\mathrm{poly}(\log(1/\varepsilon)))$.

9.3 Open Questions

1. **Optimal EML tree depth:** What is the minimum-depth EML tree for soft NAND? Our construction uses ≈ 12 levels; is this optimal? Odrzywołek [1] provides EML trees of various depths for different primitives, and the question of minimal tree depth for a given function is related to circuit complexity in the EML model.
2. **Practical gate counts:** The $O(n^3)$ gate count for ε-NAND approximation of EML is likely not tight. Can tighter bounds be derived using Karatsuba multiplication ($O(n^{1.585})$) or FFT-based multiplication ($O(n \log n \log \log n)$)?
3. **Noise-tolerant analog circuits:** Can physical analog circuits implementing G_ε be designed with the restoration property built in? The work of Pippenger [12] and Evans & Schulman

[13] suggests that analog VLSI implementations might be feasible if the gate noise is kept below the critical threshold.

4. **Generalization to other universal primitives:** Odrzywołek [1] identified two cousins of EML: $\mathrm{edl}(x, y) = e^x / \ln y$ and $-\mathrm{eml}(y, x) = \ln x - e^y$. Does a similar duality hold for these operators with different gate relaxations? The edl operator, involving division rather than subtraction, might correspond to a different soft logic gate.
5. **Complexity of restoration:** The restoration circuit $R(x)$ uses 4 gate evaluations. Is there a more efficient restoration circuit with stronger contraction? The connection to the noise thresholds of Evans & Pippenger [14] suggests that better restoration circuits might exist, potentially with noise thresholds closer to the theoretical maximum of $1/2$.
6. **EML tree minimization:** Given a target function f and precision γ, what is the minimum-depth EML tree achieving $\|f - \hat{f}\|_\infty < \gamma$? This is the continuous analog of circuit minimization for Boolean functions and appears to be computationally hard in general.
7. **Relationship to neural network expressivity:** The EML-algebraic density result (Theorem 6.5(c)) and Cybenko's universal approximation theorem [18] both establish density of a function class generated by a single primitive operation. Can a direct equivalence be shown between the expressivity of depth-d NAND circuits and depth-d ReLU/sigmoid networks?
8. **Exact-on-Boolean assumption:** How robust are the signal restoration results if the exact-on-Boolean-inputs condition (Definition 3.1) is relaxed? This connects to the general noisy computation model of Pippenger [12].
9. **Characterization of $\overline{\mathcal{N}}$:** Is $\overline{\mathcal{N}} = C_B([0, 1]^n, [0, 1])$? The Boolean-at-corners property is a necessary condition for membership in $\overline{\mathcal{N}}$; whether it is also sufficient remains open. A positive answer would provide a complete characterization of the NAND-algebra's closure and strengthen the discrete side of the duality.
10. **Exploiting the Boolean-at-corners property:** Rather than viewing the corner constraint as a limitation, can it be exploited constructively? Functions in $\overline{\mathcal{N}}$ that are Boolean at corners interpolate naturally between discrete Boolean logic and continuous computation, suggesting a deeper algebraic structure at the discrete–continuous interface.

9.4 Conclusion

We have constructed an explicit mathematical bridge between the EML operator of Odrzywołek [1] — the "Sheffer stroke of continuous mathematics" — and the NAND gate of Boolean logic. The bridge consists of four transitions: an exact algebraic identity (EML $\to$ NAND), a domain relaxation with bounded noise (NAND $\to$ ε-NAND), a constructive circuit implementation (ε-NAND $\to$ Approx EML), and a uniform limiting argument (Approx EML $\to$ EML). The key technical tool is the Signal Restoration Theorem, which shows that the nonlinearity inherent in NAND provides intrinsic error correction with a contraction factor of $\delta \mapsto 4\delta^2 + 4\varepsilon$ and a fixed-point precision of $\Theta(\varepsilon)$.

The duality reveals a deep structural unity: both the discrete and continuous worlds achieve computational universality through a single binary primitive, and in both cases, the universal operation carries within itself the seeds of error resilience. The continuous Sheffer stroke $\mathrm{eml}(x, y) = e^x - \ln y$ and the discrete Sheffer stroke $\mathrm{NAND}(a, b) = \neg(a \wedge b)$ are not merely analogous — they are constructively equivalent through the framework developed here. The crucial difference lies in the nature of universality: Boolean NAND achieves *exact* finite representation of all functions in its class, while EML achieves *approximate* finite representation of all functions in its class, with the uniform closure being the appropriate notion of completeness for the continuous setting.

References

[1] Odrzywołek, A. (2026). All elementary functions from a single binary operator. *arXiv preprint* arXiv:2603.21852 [cs.SC]. https://arxiv.org/abs/2603.21852

[2] Sheffer, H. M. (1913). A set of five independent postulates for Boolean algebras, with application to logical constants. *Transactions of the American Mathematical Society*, 14(4), 481–488. https://doi.org/10.1090/S0002-9947-1913-1500960-1

[3] Post, E. L. (1941). *The Two-Valued Iterative Systems of Mathematical Logic*. Annals of Mathematics Studies, No. 5. Princeton University Press. https://doi.org/10.2307/2268608

[4] Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338–353. https://doi.org/10.1016/S0019-9958(65)90241-X

[5] Klement, E. P., Mesiar, R., & Pap, E. (2000). *Triangular Norms*. Trends in Logic, Vol. 8. Kluwer Academic Publishers. https://doi.org/10.1007/978-94-015-9541-1

[6] Shannon, C. E. (1941). Mathematical theory of the differential analyzer. *Journal of Mathematics and Physics*, 20, 337–354. https://doi.org/10.1002/sapm1941201337

[7] Pour-El, M. B. (1974). Abstract computability and its relation to the general purpose analog computer (some connections between logic, differential equations, and analog computers). *Transactions of the American Mathematical Society*, 199, 1–28. https://doi.org/10.1090/S0002-9947-1974-0347575-X

[8] Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. https://doi.org/10.1016/0893-6080(89)90020-8

[9] Stone, M. H. (1937). Applications of the theory of Boolean rings to general topology. *Transactions of the American Mathematical Society*, 41(3), 375–481. https://doi.org/10.1090/S0002-9947-1937-1501905-7

[10] Stone, M. H. (1948). The generalized Weierstrass approximation theorem. *Mathematics Magazine*, 21(4), 167–184 and 21(5), 237–254. https://doi.org/10.2307/3029750

[11] Von Neumann, J. (1956). Probabilistic logics and the synthesis of reliable organisms from unreliable components. In C. E. Shannon & J. McCarthy (Eds.), *Automata Studies* (pp. 43–98). Annals of Mathematics Studies, No. 34. Princeton University Press. https://doi.org/10.1515/9781400882618-003

[12] Pippenger, N. (1985). On networks of noisy gates. *26th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 30–38. IEEE. https://doi.org/10.1109/SFCS.1985.31

[13] Evans, W. S. & Schulman, L. J. (1999). Signal propagation and noisy circuits. *IEEE Transactions on Information Theory*, 45(7), 2367–2373. https://doi.org/10.1109/18.796377

[14] Evans, W. & Pippenger, N. (1998). On the maximum tolerable noise for reliable computation by formulas. *IEEE Transactions on Information Theory*, 44(3), 1299–1305. https://doi.org/10.1109/18.669420

[15] Volder, J. E. (1959). The CORDIC trigonometric computing technique. *IRE Transactions on Electronic Computers*, EC-8(3), 330–334. https://doi.org/10.1109/TEC.1959.5222693

[16] Walther, J. S. (1971). A unified algorithm for elementary functions. *AFIPS Conference Proceedings, 1971 Spring Joint Computer Conference*, Vol. 38, pp. 379–385. https://doi.org/10.1145/1478786.1478840

[17] Strassen, V. (1969). Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4), 354–356. https://doi.org/10.1007/BF02165411

[18] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314. https://doi.org/10.1007/BF02551274

[19] Kolmogorov, A. N. (1957). On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. *Doklady Akademii Nauk SSSR*, 114, 953–956.

[20] Bournez, O. & Campagnolo, M. L. (2006). The general purpose analog computer and computable analysis are two equivalent paradigms of analog computation. *TAMC 2006*, Lecture Notes in Computer Science, Vol. 3959, Springer, pp. 631–643. https://doi.org/10.1007/11750321_60

[21] Pippenger, N. (1989). Invariance of complexity measures for networks with unreliable gates. *Journal of the ACM*, 36(3), 531–539. https://doi.org/10.1145/65950.77248

[22] Weierstrass, K. (1885). Über die analytische Darstellbarkeit sogenannter willkürlicher Functionen einer reellen Veränderlichen. *Sitzungsberichte der Königlich Preußischen Akademie der Wissenschaften zu Berlin*, Part 2: 633–639 and 789–805.

[23] Bournez, O. & Pouly, A. (2021). A survey on analog models of computation. In: *Handbook of Computability and Complexity in Analysis*, Springer. arXiv:1805.05729. https://arxiv.org/abs/1805.05729